

Ejercicio 1

Recuperación: Con esta prueba vemos la capacidad de recuperación del sistema tras un error

Seguridad: Con esta prueba, vemos la seguridad que tiene el sistema

Estrés: Con esta prueba, llevamos al sistema y sus componentes al límite para ver sus capacidades límites (Válgase la redundancia)

Rendimiento: Con esta prueba vemos el tiempo de respuesta del sistema

Unitarias: Con esta prueba: verificamos el comportamiento correcto de bloques de código

Regresión: Con esta prueba, verificamos el comportamiento del sistema al efectuar ciertos cambios en el código

Integración: Con esta prueba, metemos los cambios hechos y verificamos que no surgen nuevos errores

Ejercicio 2

Herramientas de depuración:

- **Debug File:** Nos permite ver el comportamiento del código a tiempo real
- **Breakpoints:** Punto que marcas sobre una línea para que el código se pare/termine sobre esta
- **Step Over/ Step Into:** Se hace dentro del debugger, con el step over nos saltamos una línea y con el step into nos metemos en lo que vale una variable
- **Menú de variables:** Es una herramienta que nos permite saber cuanto vale cada variable y permite modificar el valor de alguna para hacer determinadas pruebas

Pruebas de caja blanca: Son las pruebas donde tenemos acceso a ver el comportamiento del código, es decir, el resultado, no nos abstraemos

Pruebas de caja negra: Son las pruebas donde tenemos que abstraernos del código, estas, se realizan sobre la interfaz, abstrayéndonos del código

Ejercicio 3

Caso límite (Damos 0 y devuelve 1):

@Test

```
public void testFactorial() throws Exception {
```

```
    System.out.println("factorial");
```

```
    int num = 0;
```

```

Factorial instance = new Factorial();

int expResult = 1;

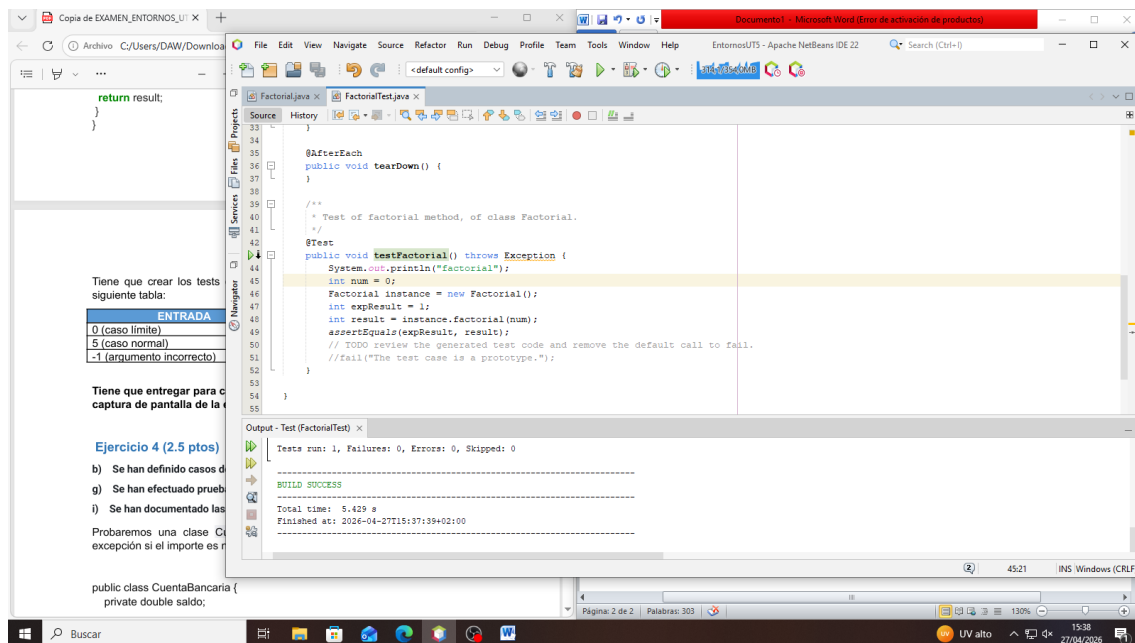
int result = instance.factorial(num);

assertEquals(expResult, result);

// TODO review the generated test code and remove the default call to fail.

//fail("The test case is a prototype.");
}

```



Caso normal (Damos 5, devuelve 120)

@Test

```

public void testFactorial() throws Exception {

    System.out.println("factorial");

    int num = 5;

    Factorial instance = new Factorial();

    int expResult = 120;

    int result = instance.factorial(num);

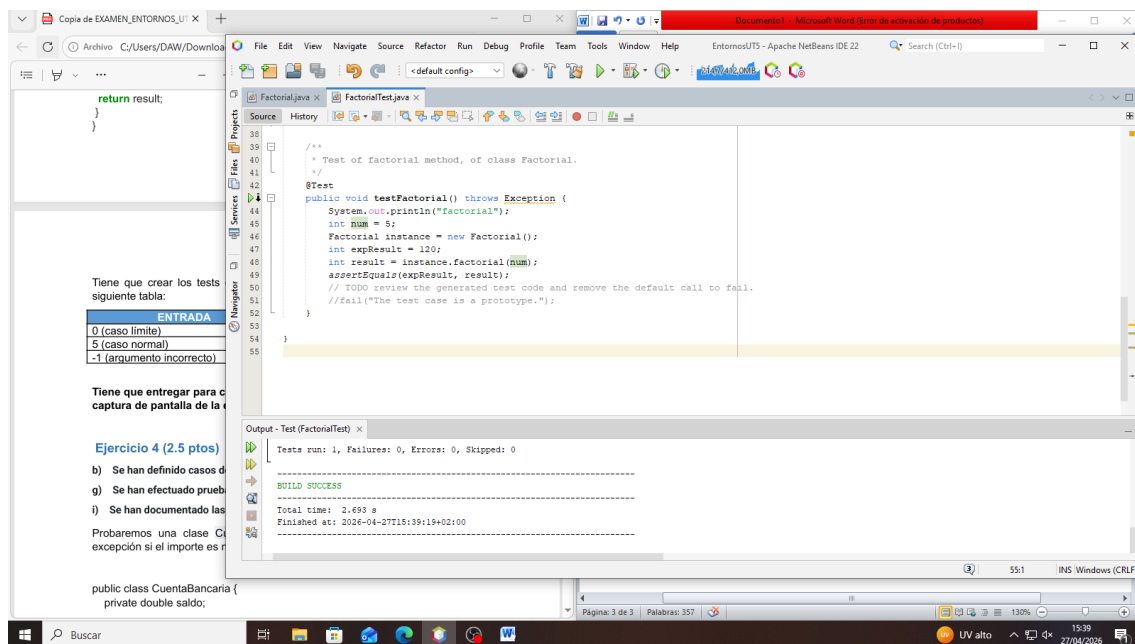
    assertEquals(expResult, result);

    // TODO review the generated test code and remove the default call to fail.

    //fail("The test case is a prototype.");
}

```

}



Caso argumento incorrecto (Damos -1, devuelve excepción)

@Test

```
public void testFactorial() throws Exception {
```

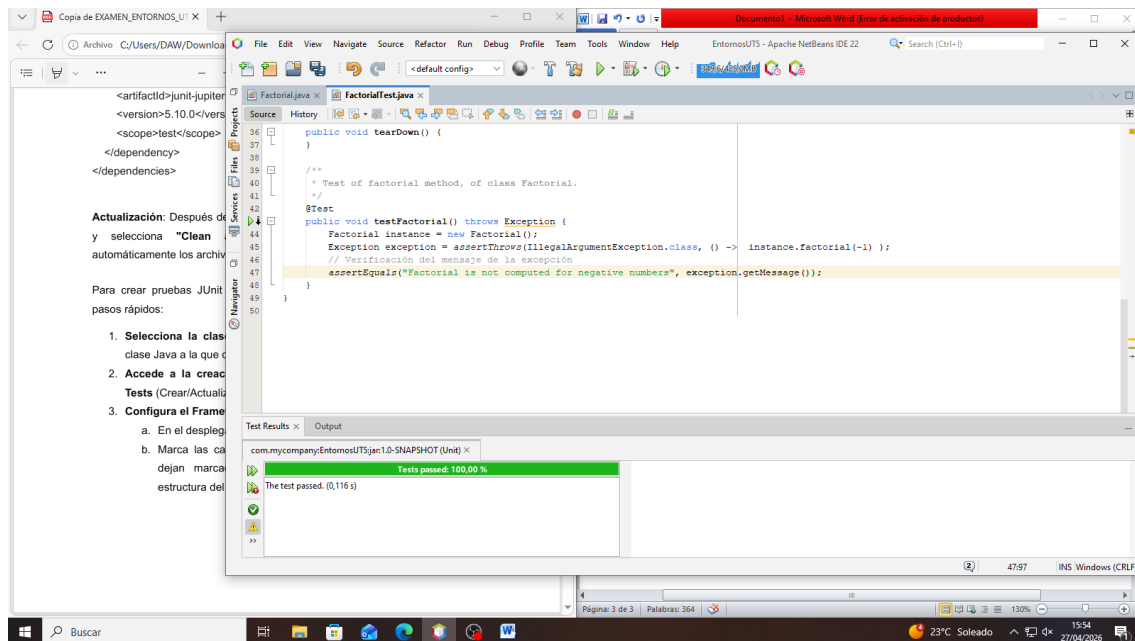
```
    Factorial instance = new Factorial();
```

```
    Exception exception = assertThrows(IllegalArgumentException.class, () ->  
    instance.factorial(-1) );
```

```
    // Verificación del mensaje de la excepción
```

```
    assertEquals("Factorial is not computed for negative numbers", exception.getMessage());
```

```
}
```



Ejercicio 4

Saldo inicial 100, retira -50

@Test

```
public void testRetirar() {
```

```
    System.out.println("retirar");
```

```
    double cantidad = 100;
```

```
    CuentaBancaria instance = new CuentaBancaria(cantidad);
```

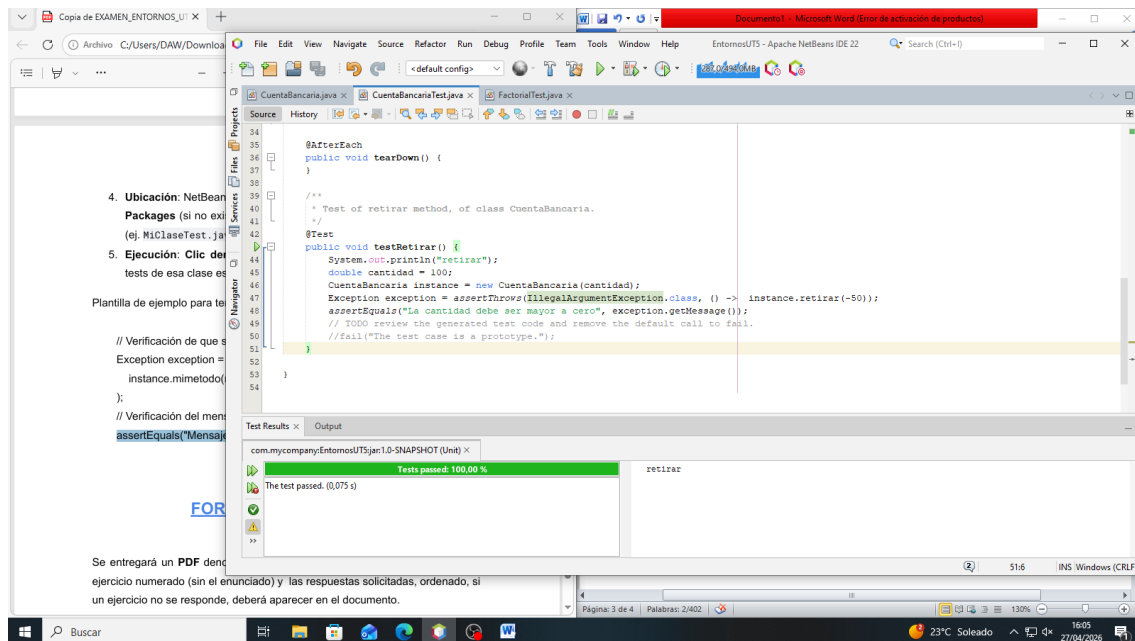
```
    Exception exception = assertThrows(IllegalArgumentException.class, () ->
    instance.retirar(-50));
```

```
    assertEquals("La cantidad debe ser mayor a cero", exception.getMessage());
```

```
    // TODO review the generated test code and remove the default call to fail.
```

```
    //fail("The test case is a prototype.");
```

```
}
```



Saldo inicial 100, retira 150

@Test

```
public void testRetirar() {
```

```
    System.out.println("retirar");
```

```
    double cantidad = 100;
```

```
    CuentaBancaria instance = new CuentaBancaria(cantidad);
```

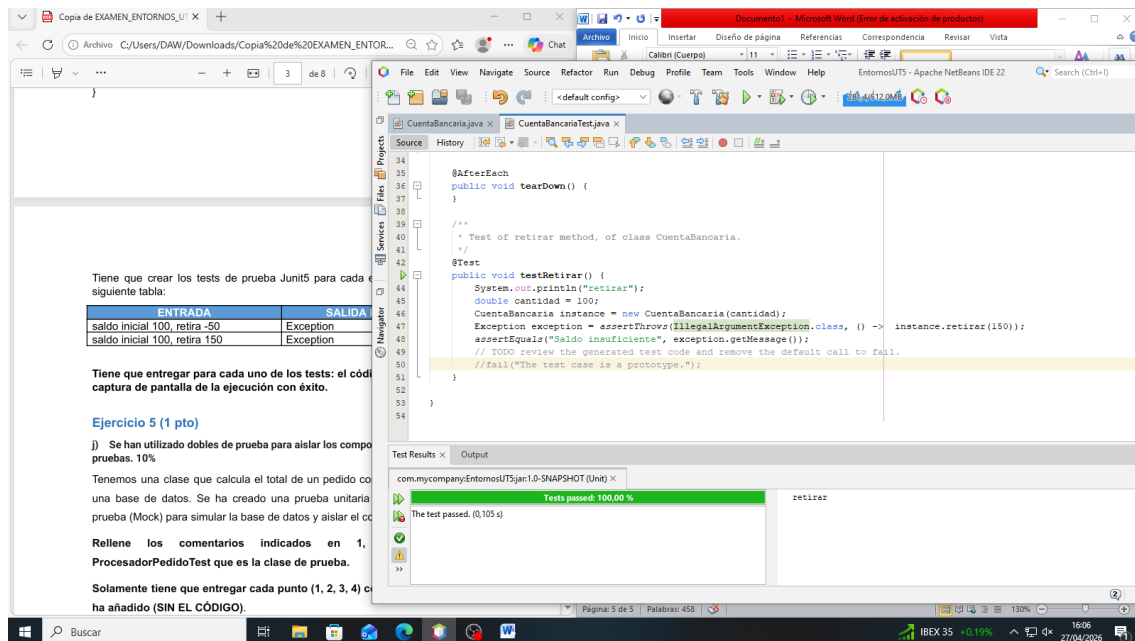
```
    Exception exception = assertThrows(IllegalArgumentException.class, () ->
    instance.retirar(150));
```

```
    assertEquals("Saldo insuficiente", exception.getMessage());
```

```
    // TODO review the generated test code and remove the default call to fail.
```

```
    //fail("The test case is a prototype.");
```

```
}
```



Ejercicio 5

@Test void testCalcularTotalConMock() {

// 1. CREACIÓN DEL MOCK: No hay BD real, con mock, hacemos una implementación vacía***

Inventario inventarioMock = mock(Inventario.class);

// 2. STUBBING (Definir comportamiento): Le decimos que cuando pregunte por el portátil (Laptop), nos devuelva el precio (1000)

when(inventarioMock.getPrecio("Laptop")).thenReturn(1000.0);

// 3. EJECUCIÓN: Inyectamos Mock y probamos el código

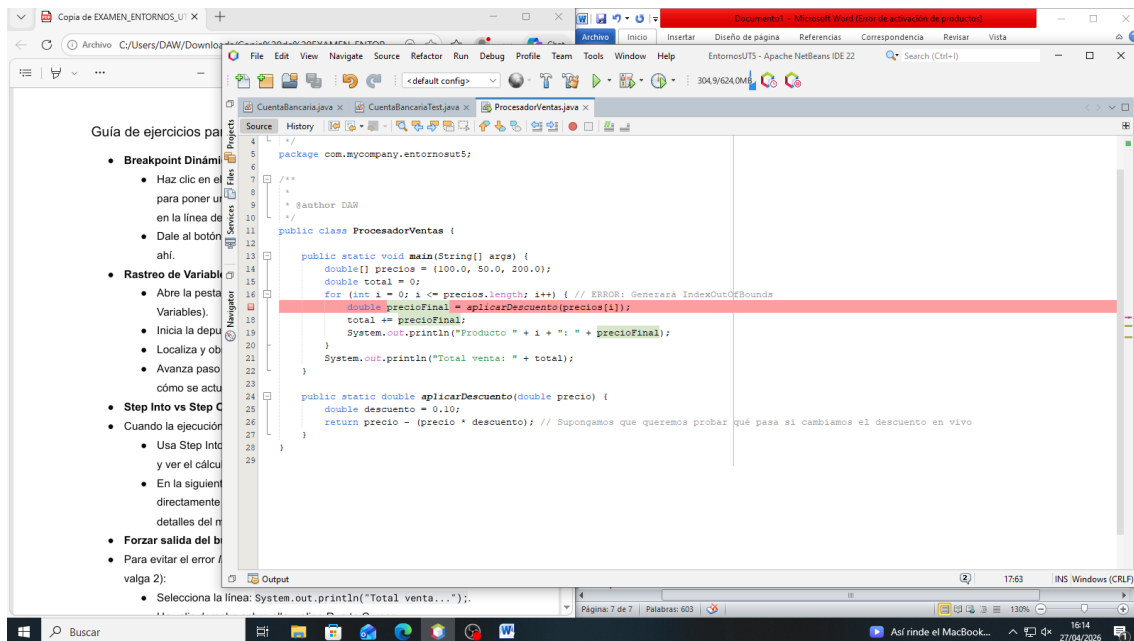
ProcesadorPedido procesador = new ProcesadorPedido(inventarioMock); double resultado = procesador.calcularTotal("Laptop", 2); assertEquals(2000.0, resultado, "El cálculo debería ser 1000 * 2");

// 4. VERIFICACIÓN: Verificamos lo que debería de dar el Mock

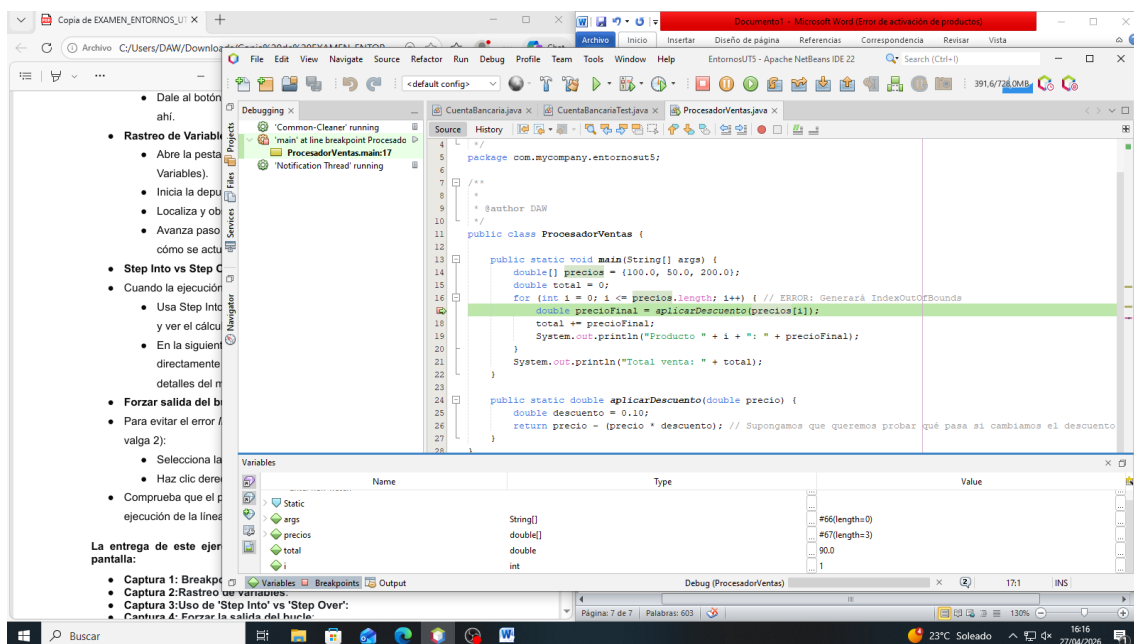
verify(inventarioMock).getPrecio("Laptop"); }

Ejercicio 6

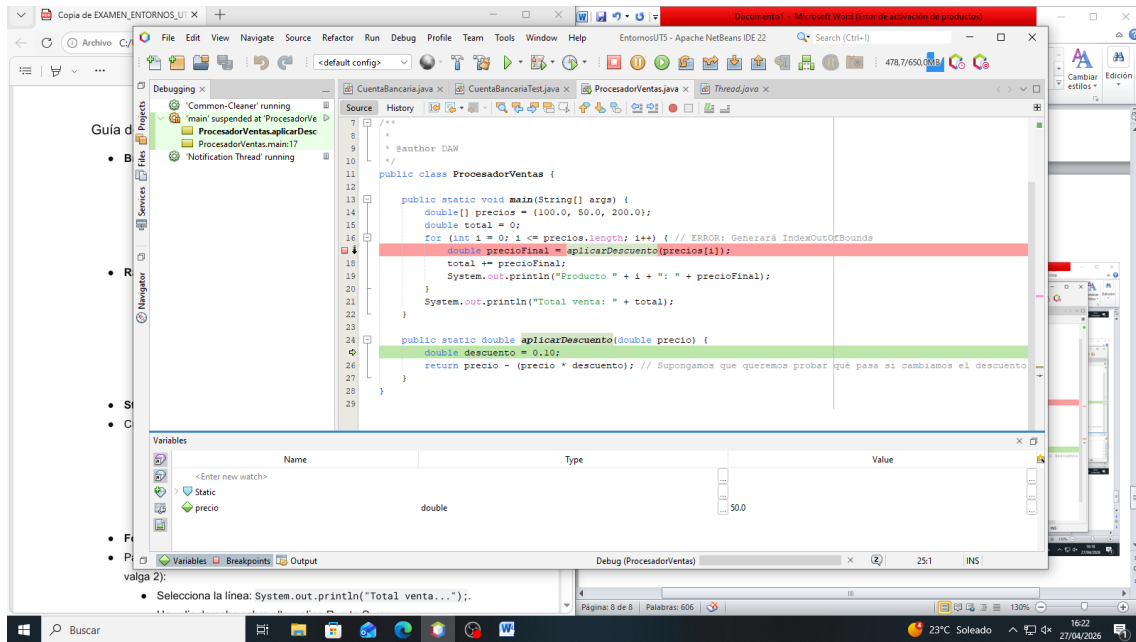
Captura Breakpoint



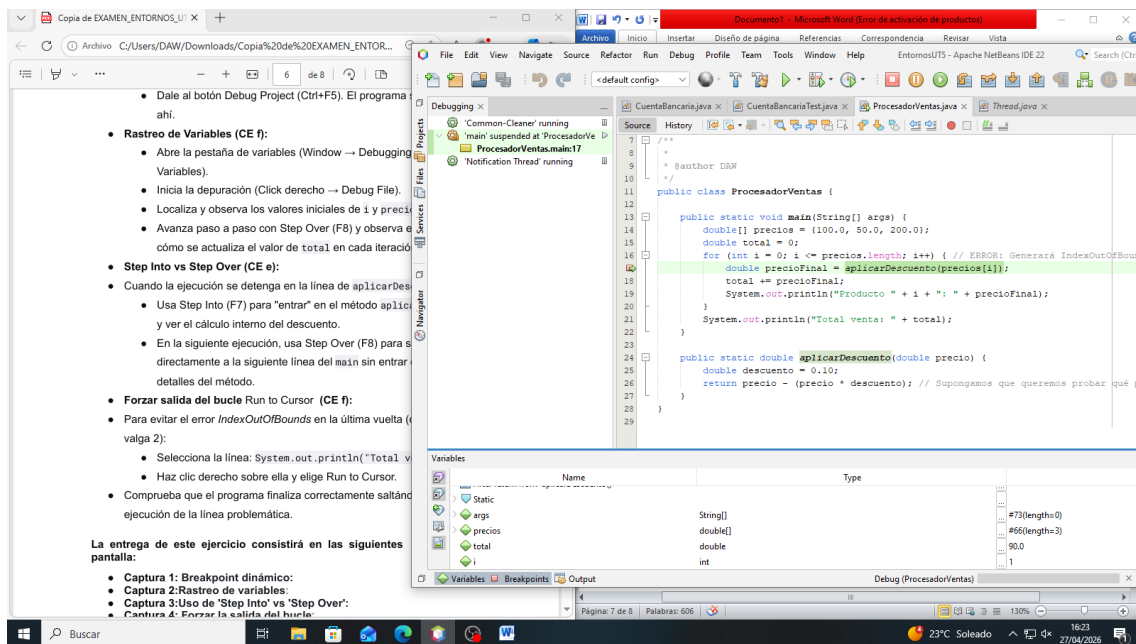
Captura Rastreo de Variables



Captura Step Into



Captura Step Over (He saltado el método)



Captura 4: Forzar salida del bucle (No me ha funcionado)

The screenshot shows an IDE with a Java project. The main window displays the source code of a class named `ProcesadorVentas`. The code includes a `main` method that initializes an array of prices, a total variable, and a loop that iterates over the prices, applying a discount and printing the total. The debugger window is open, showing the current state of the program. The `Variables` pane shows the following variables:

Name	Type	Value
Static		
args	String[]	
precios	double[]	#73[length: 66]
total	double	135.0
i	int	2

The `Output` pane shows the following output:

```
Common-Cleaner running
'main' at line breakpoint Procesado
Notification Thread running
[Product: 1, Price: 100.0, Discount: 0.10, Total: 90.0]
[Product: 2, Price: 50.0, Discount: 0.10, Total: 135.0]
[Product: 3, Price: 200.0, Discount: 0.10, Total: 135.0]
Total venta: 135.0
```

The `Debug (Procesador/Ventas)` pane shows the current state of the program, including the current line of code and the values of the variables.

La entrega de este ejercicio consistirá en las siguientes capturas de pantalla:

- Captura 1: Breakpoint dinámico:
- Captura 2: Rastreo de variables:
- Captura 3: Uso de 'Step Into' vs 'Step Over':
- Captura 4: Forzar la salida del bucle